

R Notebook

Wave 1 pilot

```
library(pacman)
```

```
p_load(readr)
```

```
# Load the data
```

```
df <- read_csv("D:/Dropbox/USD/Dissertation/Collab folder/Data/Dissertation - Pilot_May 1, 2025_16.58.csv")
```

```
## Rows: 43 Columns: 32
```

```
## -- Column specification -----
```

```
## Delimiter: ","
```

```
## chr (23): Start Date, End Date, Response Type, IP Address, Recorded Date, Re...
```

```
## dbl (4): Progress, Duration (in seconds), Location Latitude, Location Longi...
```

```
## lgl (5): Finished, Recipient Last Name, Recipient First Name, Recipient Ema...
```

```
##
```

```
## i Use 'spec()' to retrieve the full column specification for this data.
```

```
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

```
# Load the data (no need to skip rows - metadata already removed)
```

```
# df <- read.csv("Dissertation - Pilot_May 1, 2025_16.58.csv")
```

```
# Map your real column names (these were the last 11 columns before scoring)
```

```
# Adjust these column names if they have been changed slightly since upload
```

```
quiz_cols <- colnames(df)[(ncol(df) - 10):(ncol(df) - 1)]
```

```
# Recode each question as 1 (correct) or 0 (incorrect), handling NAs
```

```
df$Q1_score <- ifelse(trimws(df[[quiz_cols[1]]]) == "Contact Center", 1, 0)
```

```
df$Q2_score <- ifelse(trimws(df[[quiz_cols[2]]]) == "PCS", 1, 0)
```

```
df$Q3_score <- ifelse(trimws(df[[quiz_cols[3]]]) == "All Healthy", 1, 0)
```

```
df$Q4_score <- ifelse(trimws(df[[quiz_cols[4]]]) == "Help Desk / Support", 1, 0)
```

```
df$Q5_score <- ifelse(trimws(df[[quiz_cols[5]]]) == "Firewall issue (USC)", 1, 0)
```

```
df$Q6_score <- ifelse(trimws(df[[quiz_cols[6]]]) == "Fluctuating warnings", 1, 0)
```

```
df$Q7_score <- ifelse(trimws(df[[quiz_cols[7]]]) == "TruCare Authorization issue", 1, 0)
```

```
df$Q8_score <- ifelse(trimws(df[[quiz_cols[8]]]) == "Access to Care Alerts", 1, 0)
```

```
df$Q9_score <- ifelse(trimws(df[[quiz_cols[9]]]) == "Firewall Degradation", 1, 0)
```

```
df$Q10_score <- ifelse(trimws(df[[quiz_cols[10]]]) == "TruCare Authorization issue", 1, 0)
```

```
# Convert non-matches or NAs to 0 cleanly
```

```
score_cols <- paste0("Q", 1:10, "_score")
```

```
df[score_cols] <- lapply(df[score_cols], function(x) ifelse(is.na(x), 0, x))
```

```
# Calculate total score and percent score (adjusting for missing)
```

```
df$Total_Score <- rowSums(df[score_cols], na.rm = TRUE)
```

```
df$Percent_Score <- df$Total_Score / length(score_cols) * 100
```

```
# Optional: check a summary  
summary(df[c("Total_Score", "Percent_Score")])
```

```
##   Total_Score   Percent_Score  
##   Min.      :0.000   Min.      : 0.00  
##   1st Qu.:2.000   1st Qu.:20.00  
##   Median :3.000   Median :30.00  
##   Mean    :3.047   Mean    :30.47  
##   3rd Qu.:4.000   3rd Qu.:40.00  
##   Max.    :7.000   Max.    :70.00
```

```
# Load packages  
library(sjPlot)  
library(ggplot2)
```

```
## Warning: package 'ggplot2' was built under R version 4.4.3
```

```
##  
## Attaching package: 'ggplot2'
```

```
## The following object is masked from 'package:sjPlot':  
##  
##   set_theme
```

```
p_load(questionr)  
# Make sure 'group' is a factor  
df$group <- as.factor(df$Version)
```

```
# Run ANOVA using a linear model  
anova_model <- lm(Total_Score ~ group, data = df)
```

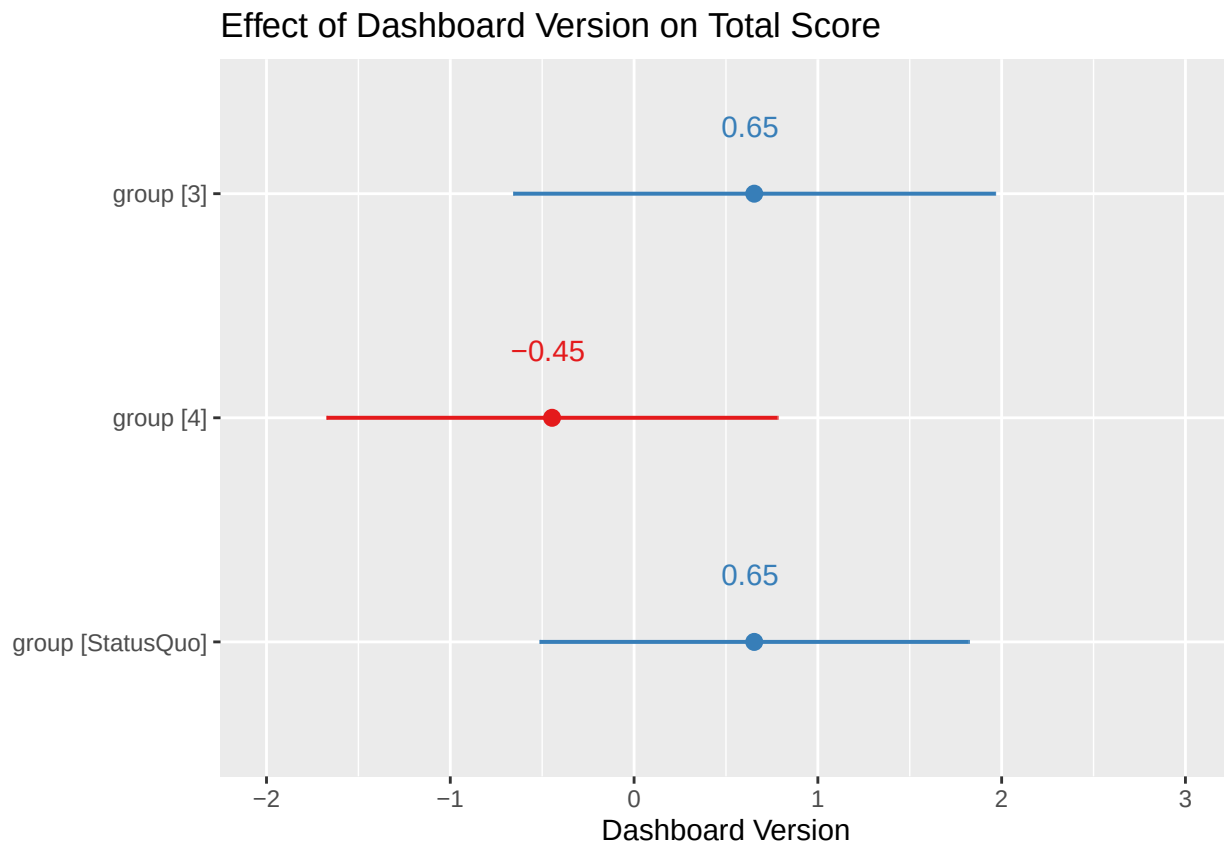
```
# Optional: inspect ANOVA table  
anova_summary <- summary(aov(anova_model))  
print(anova_summary)
```

```
##           Df Sum Sq Mean Sq F value Pr(>F)  
## group      3   8.81   2.938   1.413  0.253  
## Residuals 39  81.09   2.079
```

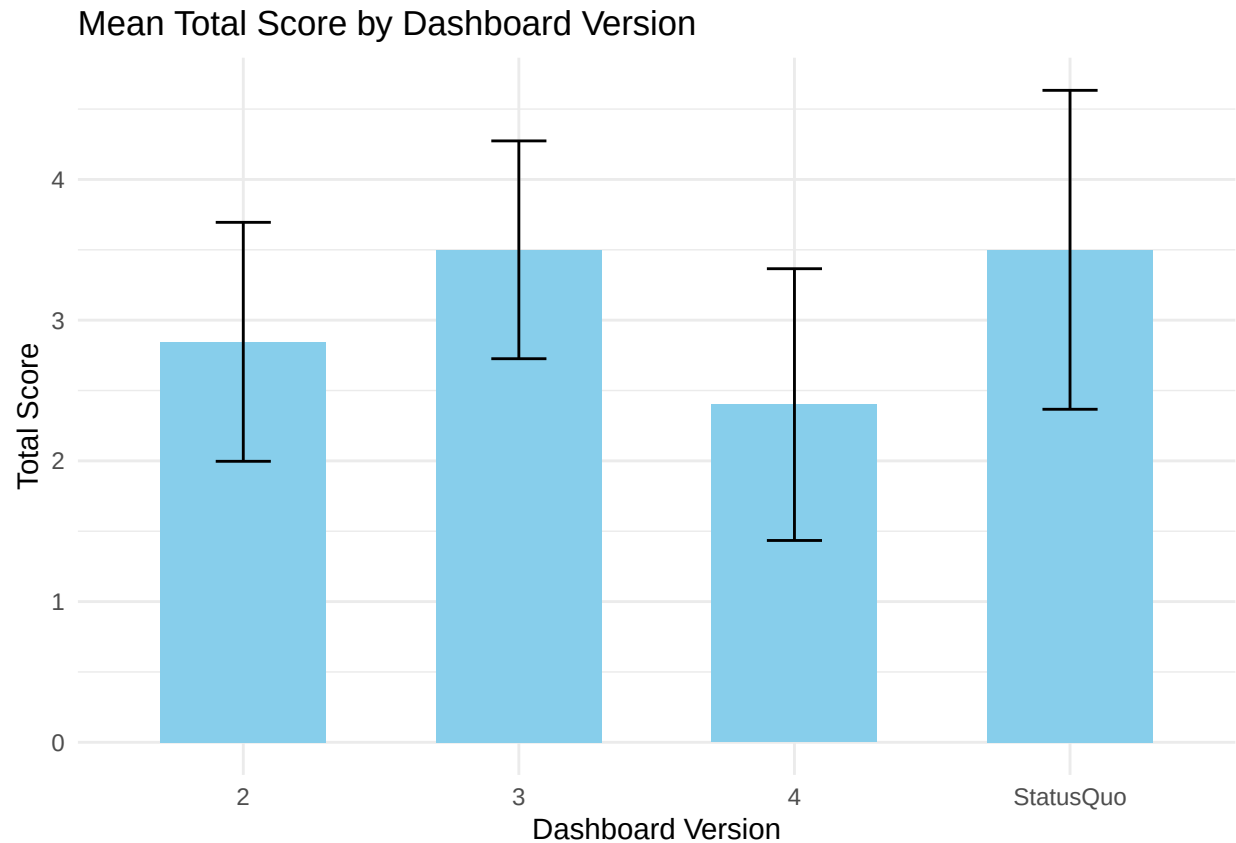
```
# Plot group effects using sjPlot  
plot_model(anova_model,  
  type = "est",          # Plot estimates  
  show.values = TRUE,  
  value.offset = .3,  
  title = "Effect of Dashboard Version on Total Score",  
  axis.title = c("Dashboard Version", "Estimated Total Score"))
```

```
## Warning: Using 'size' aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use 'linewidth' instead.
## i The deprecated feature was likely used in the sjPlot package.
## Please report the issue at <https://github.com/strengejacked/sjPlot/issues>.
## This warning is displayed once every 8 hours.
## Call 'lifecycle::last_lifecycle_warnings()' to see where this warning was
## generated.

## Warning: 'aes_string()' was deprecated in ggplot2 3.0.0.
## i Please use tidy evaluation idioms with 'aes()'.
## i See also 'vignette("ggplot2-in-packages")' for more information.
## i The deprecated feature was likely used in the sjPlot package.
## Please report the issue at <https://github.com/strengejacked/sjPlot/issues>.
## This warning is displayed once every 8 hours.
## Call 'lifecycle::last_lifecycle_warnings()' to see where this warning was
## generated.
```



```
# Optional: plot means with confidence intervals (if you want a ggplot version)
library(ggplot2)
ggplot(df, aes(x = group, y = Total_Score)) +
  stat_summary(fun = mean, geom = "bar", fill = "skyblue", width = 0.6) +
  stat_summary(fun.data = mean_cl_normal, geom = "errorbar", width = 0.2) +
  labs(title = "Mean Total Score by Dashboard Version",
       x = "Dashboard Version",
       y = "Total Score") +
  theme_minimal()
```



Wave 2 pilot

Excel steps: Remove rows 1+3, remove non-wave2 responses

Load data

```
p_load(readr)
# Load the data
df <- read_csv("D:/Dropbox/USD/Dissertation/Collab folder/Data/Dissertation+--Pilot_August+26,+2025_15.

## Rows: 924 Columns: 45
## -- Column specification -----
## Delimiter: ","
## chr (35): Start Date, End Date, Response Type, IP Address, Recorded Date, Re...
## dbl (5): Progress, Duration (in seconds), Location Latitude, Location Longi...
## lgl (5): Finished, Recipient Last Name, Recipient First Name, Recipient Ema...
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

Score data

```
# Define actual column names for the 10 quiz questions
quiz_cols <- c(
  "Which part of the organization showed the most severe issues?",
  "Which application appeared to require the most urgent attention based on its alert status?",
  "What was the overall condition of the Provider systems?",
  "Which team or role would likely be most concerned with the issues displayed in the dashboard?",
  "Based on the information shown, which incident was still being actively investigated?",
  "What trend was visible in the PCS application status over the last hour?",
  "Which issue seemed to involve a failure in external communication or handshake?",
  "Based on the dashboard layout, what kind of alert was most likely automatically generated rather than manually triggered?",
  "Which incident was in the 'Isolation' state, suggesting containment but not resolution?",
  "If you had to triage one issue first based on the dashboard, which would be the best candidate?"
)

# Score each question (keeping NA if missing)
correct_answers <- c(
  "Contact Center",
  "PCS",
  "All Healthy",
  "Help Desk / Support",
  "Firewall issue (USC)",
  "Fluctuating warnings",
  "TruCare Authorization issue",
  "Access to Care Alerts",
  "Firewall Degradation",
  "TruCare Authorization issue"
)

for(i in 1:10) {
  score_col <- paste0("Q", i, "_score")
  df[[score_col]] <- ifelse(
    is.na(df[[quiz_cols[i]]]), NA,
    ifelse(trimws(df[[quiz_cols[i]]]) == correct_answers[i], 1, 0)
  )
}

# Calculate total and percent scores (NA-safe)
score_cols <- paste0("Q", 1:10, "_score")
df$Total_Score <- rowSums(df[score_cols], na.rm = TRUE)
df$Percent_Score <- df$Total_Score / rowSums(!is.na(df[score_cols])) * 100

# Optional: summary of scores
summary(df[c("Total_Score", "Percent_Score")])
```

```
##   Total_Score   Percent_Score
##   Min.      : 0.000   Min.      : 0.0
##   1st Qu.: 2.000   1st Qu.: 20.0
##   Median : 3.000   Median : 30.0
##   Mean    : 3.083   Mean    : 37.1
##   3rd Qu.: 4.000   3rd Qu.: 50.0
##   Max.    :10.000   Max.    :100.0
```

```
## NA's :156
```

unadjusted inspection

```
# Load packages
library(sjPlot)
library(ggplot2)

# Make sure 'group' is a factor
df$group <- as.factor(df$Version)

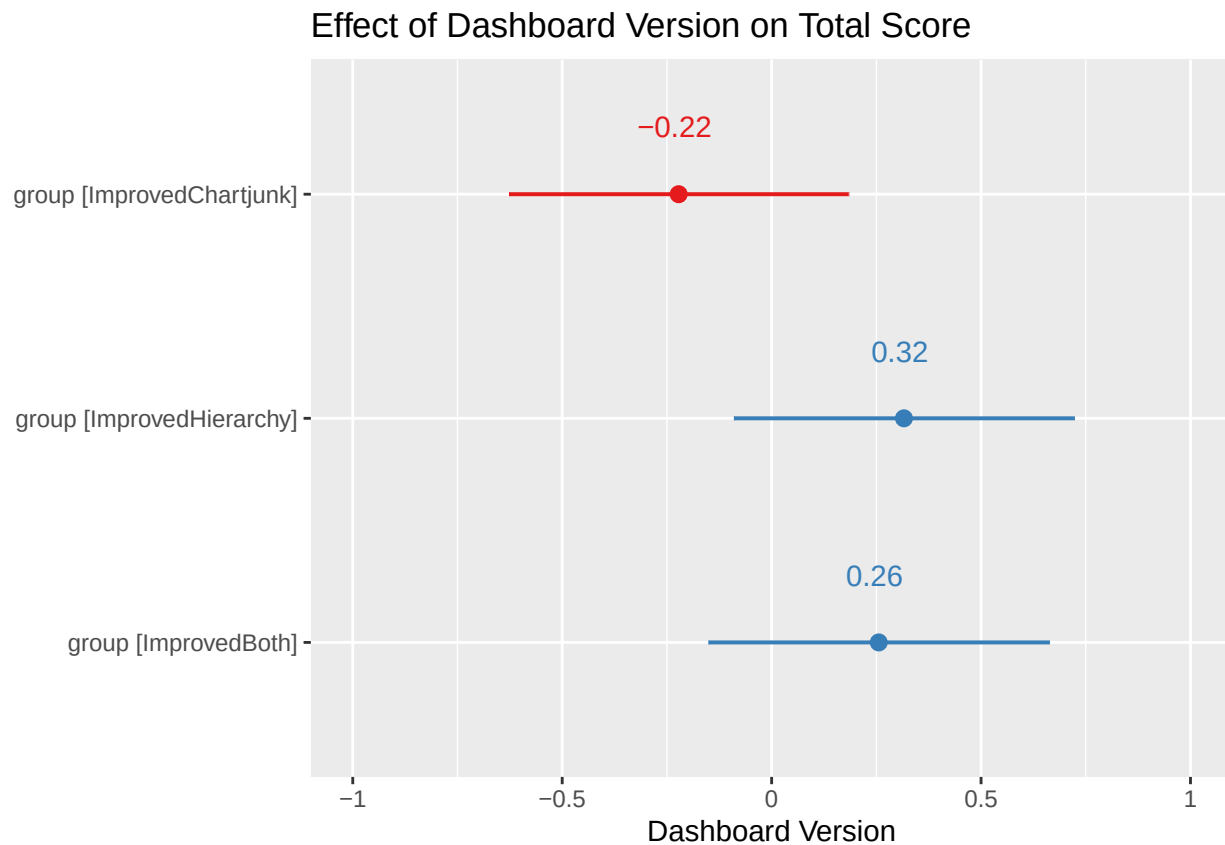
## Reordering df$group
df$group <- factor(df$group,
  levels = c("StatusQuo", "ImprovedChartjunk", "ImprovedHierarchy", "ImprovedBoth")
)

# Run ANOVA using a linear model
anova_model <- lm(Total_Score ~ group, data = df)

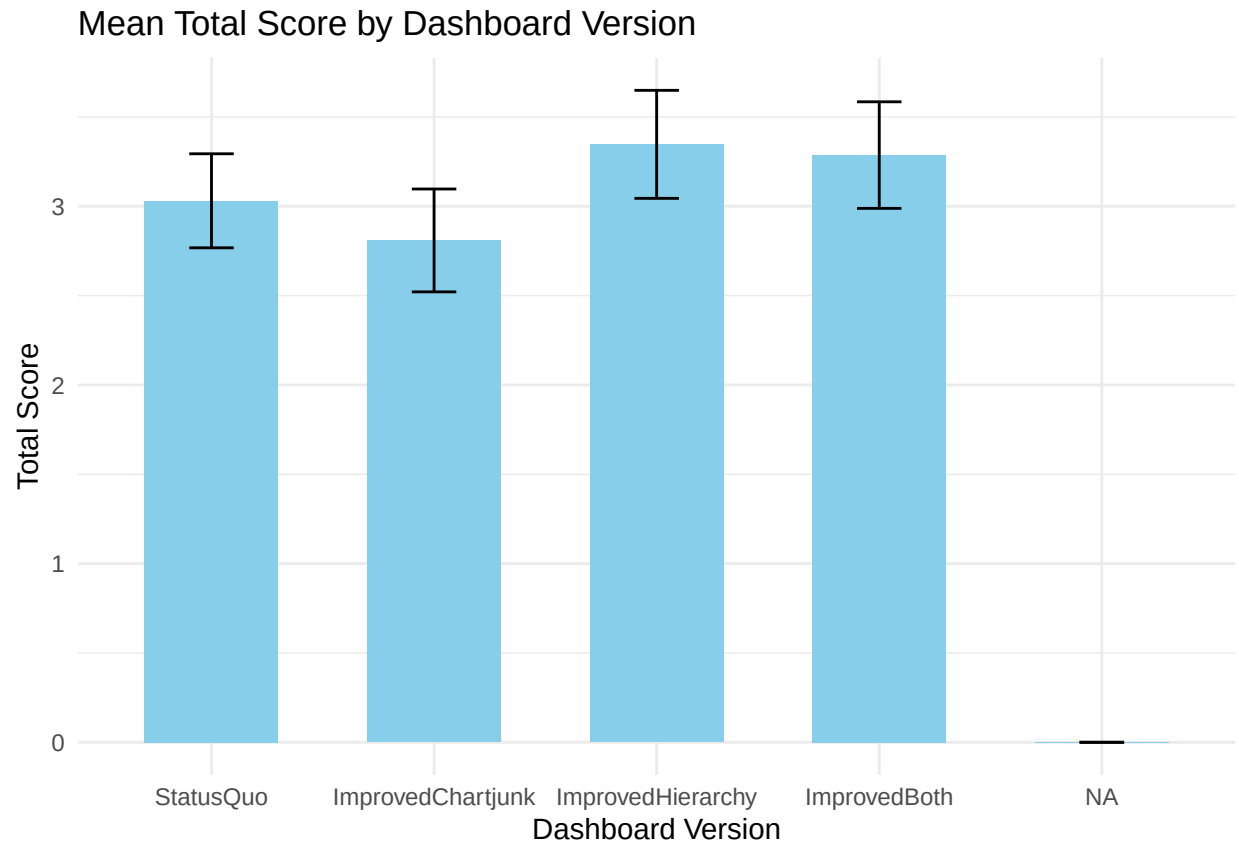
# Optional: inspect ANOVA table
anova_summary <- summary(aov(anova_model))
print(anova_summary)
```

```
##           Df Sum Sq Mean Sq F value Pr(>F)
## group      3      42  14.030    2.871 0.0355 *
## Residuals 910    4446    4.886
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## 10 observations deleted due to missingness
```

```
# Plot group effects using sjPlot
plot_model(anova_model,
  type = "est",          # Plot estimates
  show.values = TRUE,
  value.offset = .3,
  title = "Effect of Dashboard Version on Total Score",
  axis.title = c("Dashboard Version", "Estimated Total Score"))
```



```
# Optional: plot means with confidence intervals (if you want a ggplot version)
library(ggplot2)
ggplot(df, aes(x = group, y = Total_Score)) +
  stat_summary(fun = mean, geom = "bar", fill = "skyblue", width = 0.6) +
  stat_summary(fun.data = mean_cl_normal, geom = "errorbar", width = 0.2) +
  labs(title = "Mean Total Score by Dashboard Version",
       x = "Dashboard Version",
       y = "Total Score") +
  theme_minimal()
```



```
library(dplyr)
```

```
##  
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':  
##  
##   filter, lag
```

```
## The following objects are masked from 'package:base':  
##  
##   intersect, setdiff, setequal, union
```

```
# Remove respondents who got the "Help Desk / Support" question wrong (Q4)
```

```
df_valid <- df %>%
```

```
  filter(~Which team or role would likely be most concerned with the issues displayed in the dashboard?)
```

```
# Check new dataset size
```

```
nrow(df_valid)
```

```
## [1] 508
```


altered inspection

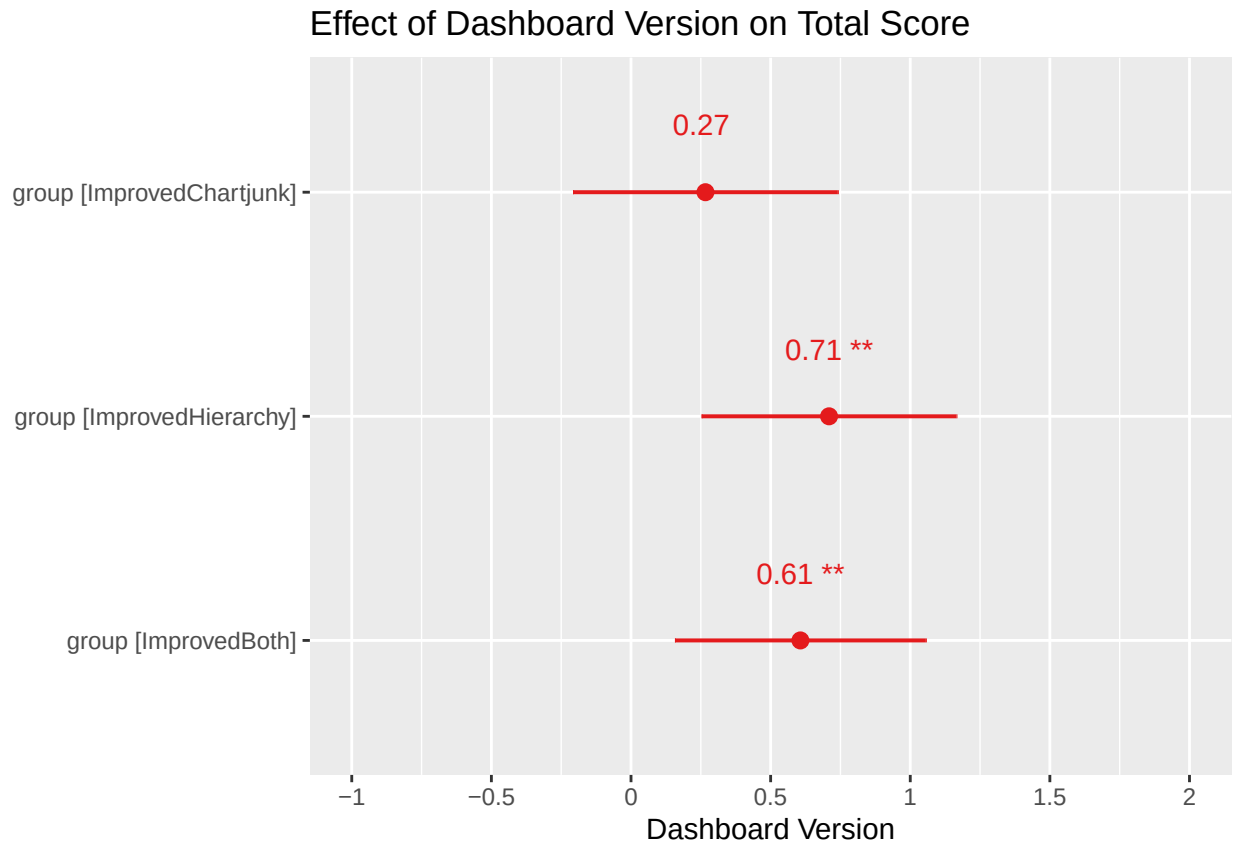
```
# Load packages
library(sjPlot)
library(ggplot2)

# Run ANOVA using a linear model
anova_model <- lm(Total_Score ~ group, data = df_valid)

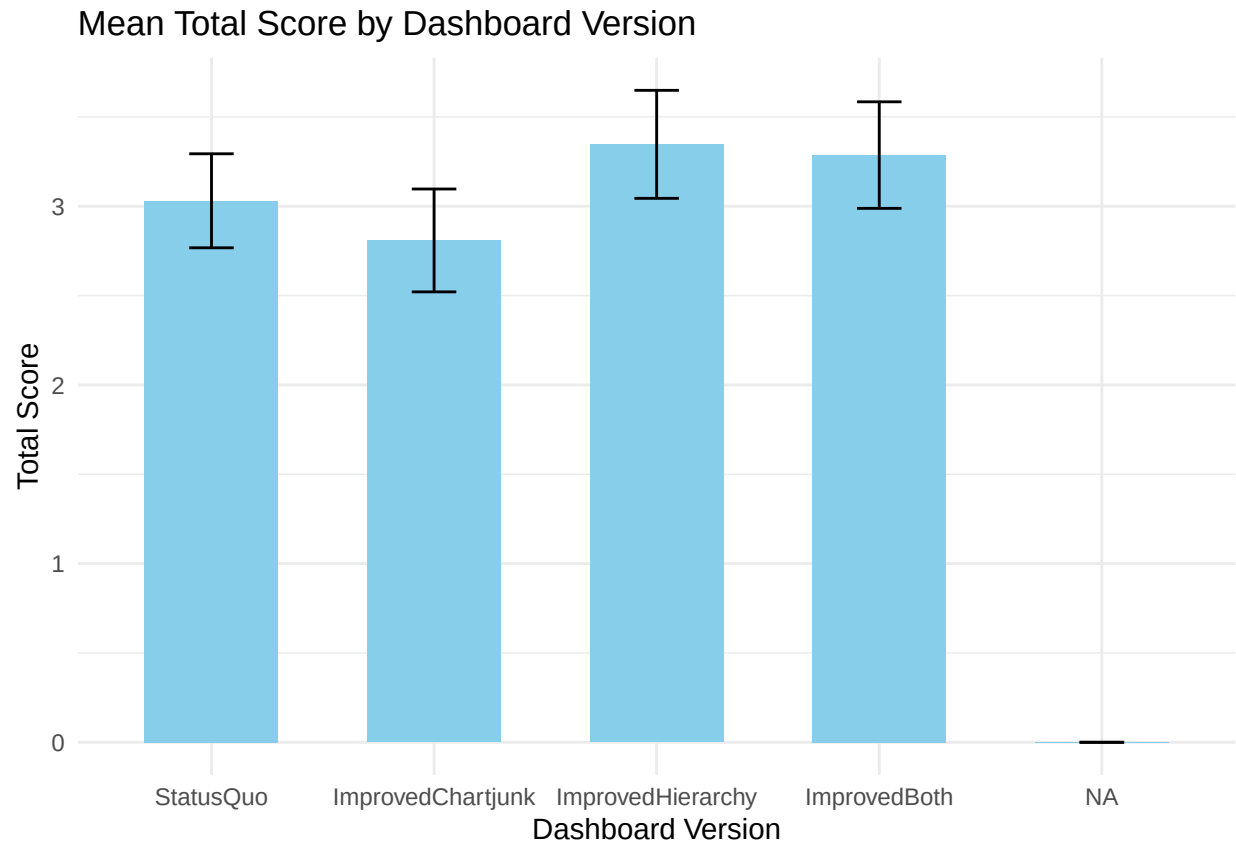
# Optional: inspect ANOVA table
anova_summary <- summary(aov(anova_model))
print(anova_summary)

##              Df Sum Sq Mean Sq F value    Pr(>F)
## group          3   41.6   13.874    3.916 0.00879 **
## Residuals    504 1785.7    3.543
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

# Plot group effects using sjPlot
plot_model(anova_model,
  type = "est",          # Plot estimates
  show.values = TRUE,
  value.offset = .3,
  title = "Effect of Dashboard Version on Total Score",
  axis.title = c("Dashboard Version", "Estimated Total Score"))
```



```
# Optional: plot means with confidence intervals (if you want a ggplot version)
library(ggplot2)
ggplot(df, aes(x = group, y = Total_Score)) +
  stat_summary(fun = mean, geom = "bar", fill = "skyblue", width = 0.6) +
  stat_summary(fun.data = mean_cl_normal, geom = "errorbar", width = 0.2) +
  labs(title = "Mean Total Score by Dashboard Version",
       x = "Dashboard Version",
       y = "Total Score") +
  theme_minimal()
```



Longstring detection

```
library(dplyr)
library(stringr)

# Define mapping of question text to display order columns
questions <- list(
  Q1 = c("Which part of the organization showed the most severe issues?",
        "Which part of the organization showed the most severe issues? - Display Order"),
  Q2 = c("Which application appeared to require the most urgent attention based on its alert status?",
        "Which application appeared to require the most urgent attention based on its alert status? - Display Order"),
  Q3 = c("What was the overall condition of the Provider systems?",
        "What was the overall condition of the Provider systems? - Display Order"),
  Q4 = c("Which team or role would likely be most concerned with the issues displayed in the dashboard?",
        "Which team or role would likely be most concerned with the issues displayed in the dashboard? - Display Order"),
  # Q5 = c("Based on the information shown, which incident was still being actively investigated?",
  #       NA), # This one has no display order column
  Q6 = c("What trend was visible in the PCS application status over the last hour?",
        "What trend was visible in the PCS application status over the last hour? - Display Order"),
  Q7 = c("Which issue seemed to involve a failure in external communication or handshake?",
        "Which issue seemed to involve a failure in external communication or handshake? - Display Order"),
  Q8 = c("Based on the dashboard layout, what kind of alert was most likely automatically generated?",
        "Based on the dashboard layout, what kind of alert was most likely automatically generated? - Display Order")
)
```

```

Q9 = c("Which incident was in the 'Isolation' state, suggesting containment but not resolution?",
       "Which incident was in the 'Isolation' state, suggesting containment but not resolution? - Dis")
Q10 = c("If you had to triage one issue first based on the dashboard, which would be the best candidate?",
        "If you had to triage one issue first based on the dashboard, which would be the best candidate?")
)

# Function to get numeric position of selected response
get_position <- function(response, display_order_string) {
  if (is.na(response) || is.na(display_order_string)) return(NA_integer_)
  choices <- str_trim(unlist(str_split(display_order_string, "\\|")))
  match(str_trim(response), choices)
}

# Create numeric position columns
for (q in names(questions)) {
  response_col <- questions[[q]][1]
  order_col <- questions[[q]][2]
  pos_col <- paste0(q, "_position")

  if (!is.na(order_col)) {
    df[[pos_col]] <- mapply(get_position, df[[response_col]], df[[order_col]])
  } else {
    df[[pos_col]] <- NA_integer_
  }
}

# Preview the resulting position data
df %>% select(ends_with("_position")) %>% head()

```

```

## # A tibble: 6 x 9
##   Q1_position Q2_position Q3_position Q4_position Q6_position Q7_position
##   <int>      <int>      <int>      <int>      <int>      <int>
## 1         1         3         1         3         2         1
## 2         4         3         2         4         4         4
## 3         1         1         1         1         1         1
## 4         3         4         4         1         1         4
## 5         3         3         3         1         2         1
## 6         2         2         4         2         4         1
## # i 3 more variables: Q8_position <int>, Q9_position <int>, Q10_position <int>

```

```

p_load(careless)

library(careless)
library(dplyr)

# Step 1: Select *_position columns
position_data <- df %>% select(ends_with("_position"))

# Step 2: Compute longstring values (max consecutive same responses)
df$longstring <- longstring(position_data)

# Step 3: Determine cutoff using 1.5 * IQR rule
iqr_val <- IQR(df$longstring, na.rm = TRUE)

```

```
q3_val <- quantile(df$longstring, 0.75, na.rm = TRUE)
cutoff <- q3_val + 200 * iqr_val
```

```
# Step 4: Flag and filter
df$flag_straightliner <- df$longstring > cutoff
df_cleaned <- df %>% filter(!flag_straightliner)
```

```
# Step 5 (optional): Print info
cat("Outlier threshold:", cutoff, "\n")
```

```
## Outlier threshold: 203
```

```
cat("Removed", sum(df$flag_straightliner), "participants for straightlining.\n")
```

```
## Removed 0 participants for straightlining.
```

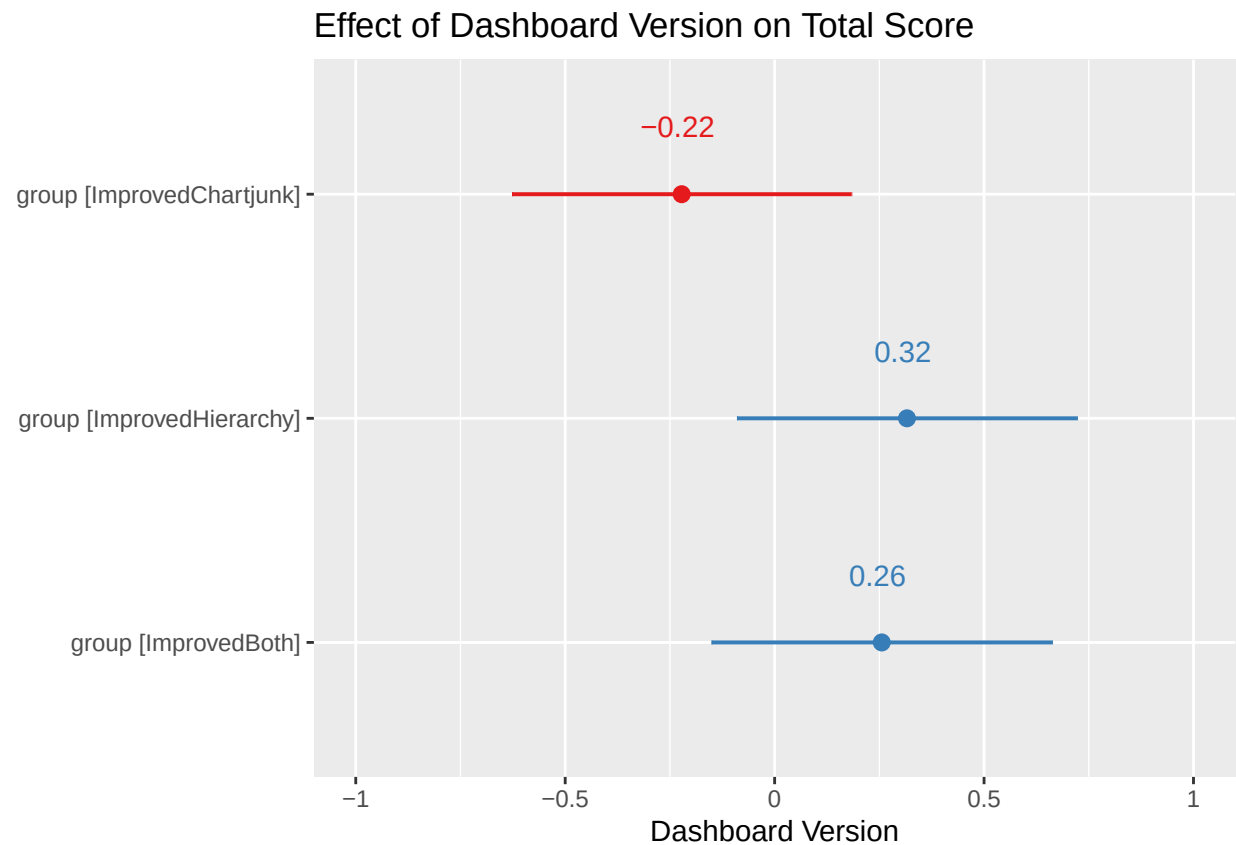
```
# Load packages
library(sjPlot)
library(ggplot2)
```

```
# Run ANOVA using a linear model
anova_model <- lm(Total_Score ~ group, data = df_cleaned)
```

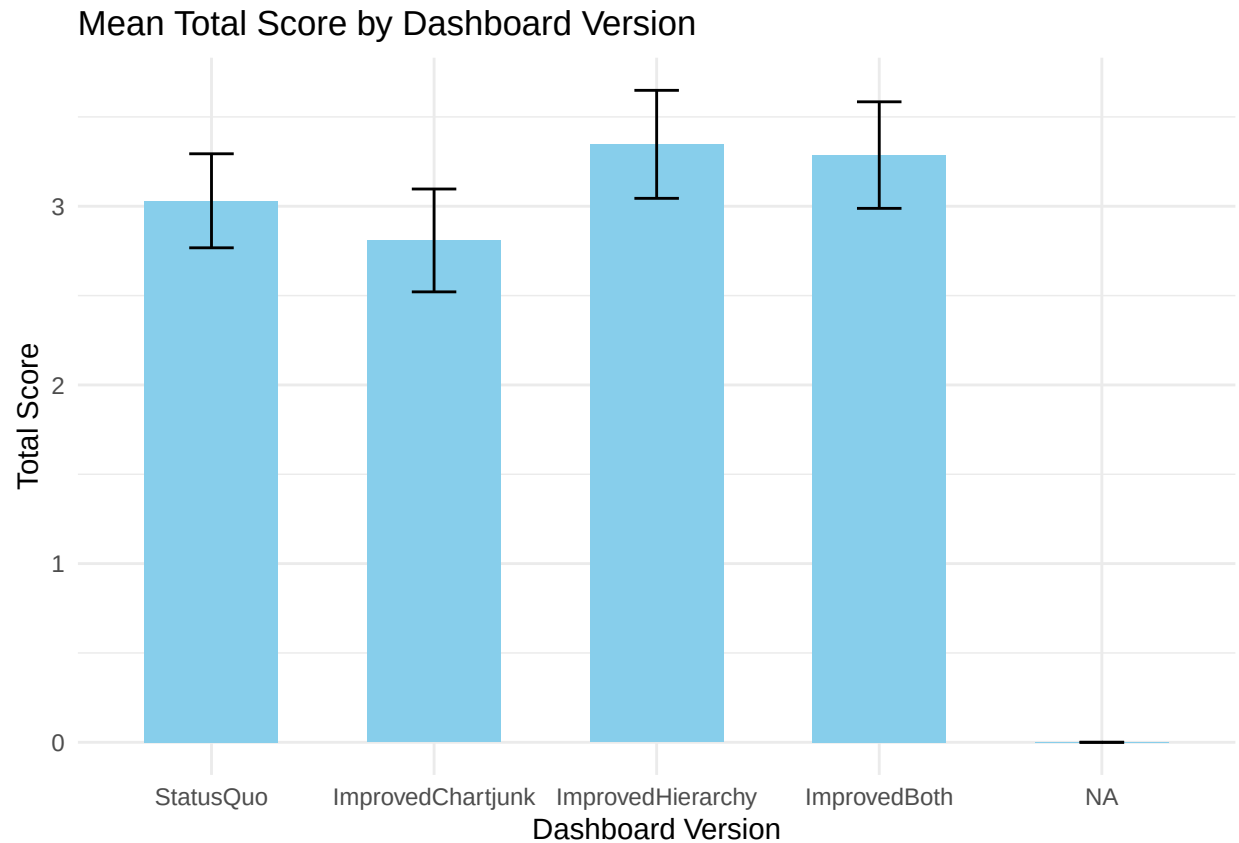
```
# Optional: inspect ANOVA table
anova_summary <- summary(aov(anova_model))
print(anova_summary)
```

```
##              Df Sum Sq Mean Sq F value Pr(>F)
## group          3     42  14.030   2.871 0.0355 *
## Residuals    910    4446    4.886
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## 10 observations deleted due to missingness
```

```
# Plot group effects using sjPlot
plot_model(anova_model,
  type = "est",          # Plot estimates
  show.values = TRUE,
  value.offset = .3,
  title = "Effect of Dashboard Version on Total Score",
  axis.title = c("Dashboard Version", "Estimated Total Score"))
```



```
# Optional: plot means with confidence intervals (if you want a ggplot version)
library(ggplot2)
ggplot(df, aes(x = group, y = Total_Score)) +
  stat_summary(fun = mean, geom = "bar", fill = "skyblue", width = 0.6) +
  stat_summary(fun.data = mean_cl_normal, geom = "errorbar", width = 0.2) +
  labs(title = "Mean Total Score by Dashboard Version",
       x = "Dashboard Version",
       y = "Total Score") +
  theme_minimal()
```



Combined careless detection

```
# Combine the rows in df_valid and df_cleaned that *should be kept*  
# First, get the IDs (assuming Response ID is your unique identifier)  
valid_ids <- df_valid$`Response ID`  
clean_ids <- df_cleaned$`Response ID`  
  
# Find those that pass both filters (intersection)  
final_ids <- intersect(valid_ids, clean_ids)  
  
# Filter original df to just those  
df_final <- df %>% filter(`Response ID` %in% final_ids)
```

counts of rejections

```
library(dplyr)  
  
# Get ID lists  
valid_ids <- df_valid$`Response ID`  
clean_ids <- df_cleaned$`Response ID`  
  
# Start from full data
```

```
df$exclusion_reason <- case_when(
  !(df$`Response ID` %in% valid_ids) & !(df$`Response ID` %in% clean_ids) ~ "Failed both",
  !(df$`Response ID` %in% valid_ids) & (df$`Response ID` %in% clean_ids) ~ "Failed validation",
  (df$`Response ID` %in% valid_ids) & !(df$`Response ID` %in% clean_ids) ~ "Failed longstring",
  TRUE ~ "Passed both"
)

# View counts
table(df$exclusion_reason)
```

```
##
## Failed validation      Passed both
##           416           508
```

Inspect again

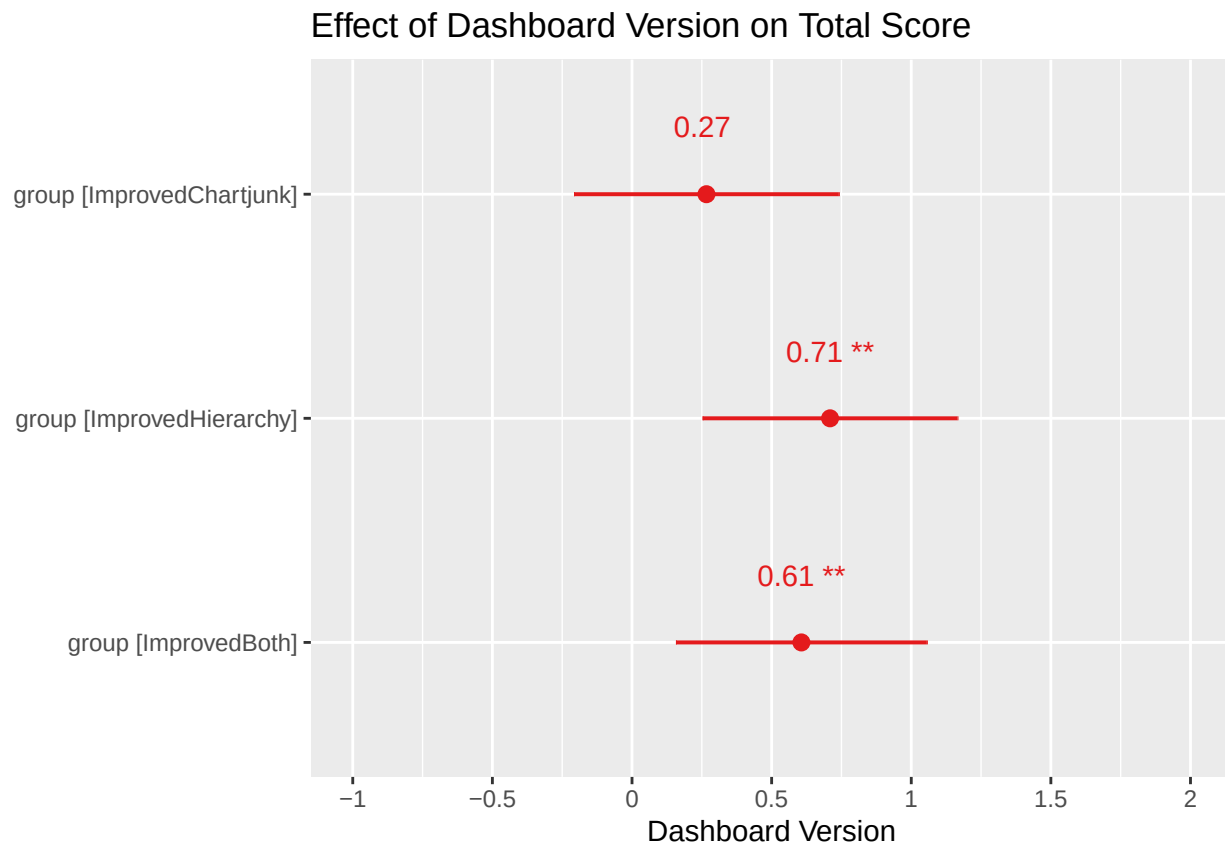
```
library(sjPlot)
library(ggplot2)

# Run ANOVA using a linear model
anova_model <- lm(Total_Score ~ group, data = df_final)

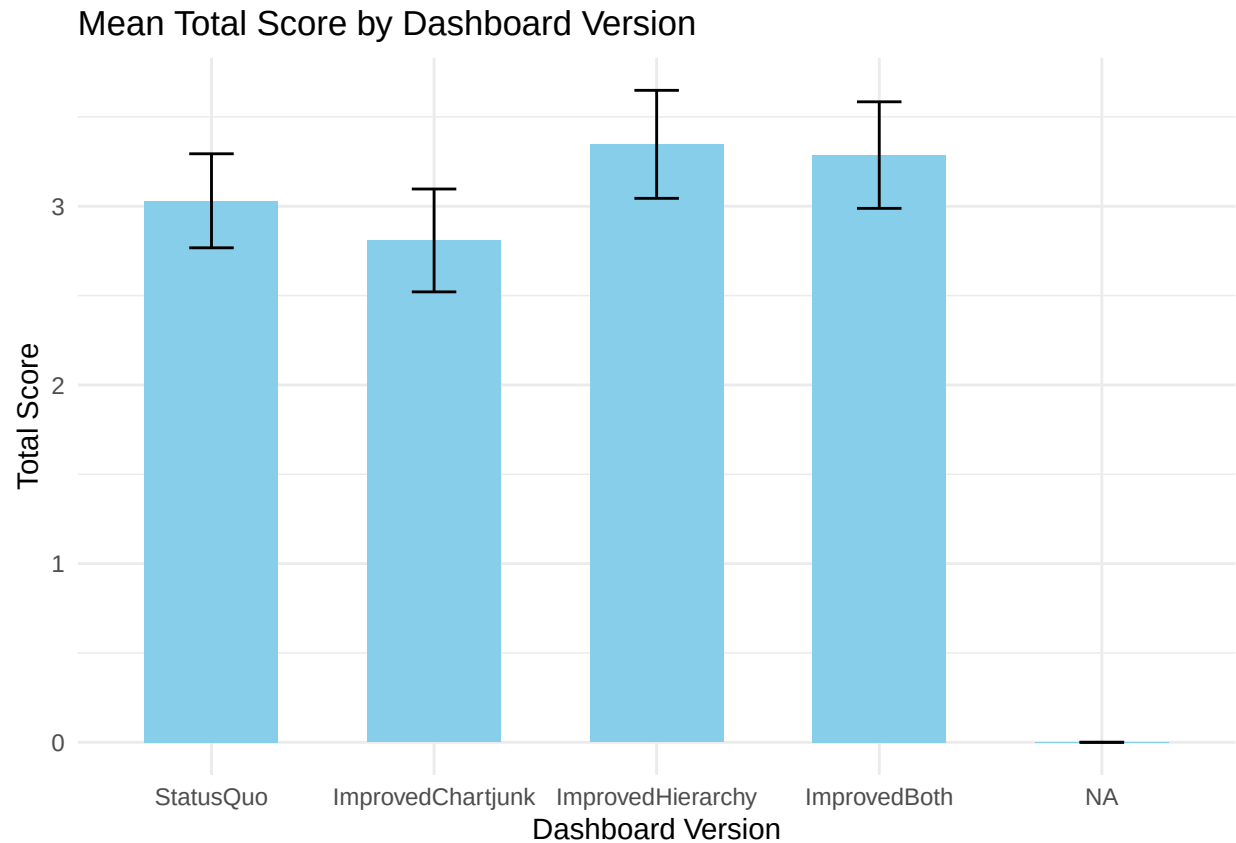
# Optional: inspect ANOVA table
anova_summary <- summary(aov(anova_model))
print(anova_summary)
```

```
##              Df Sum Sq Mean Sq F value    Pr(>F)
## group          3   41.6   13.874    3.916 0.00879 **
## Residuals    504 1785.7    3.543
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
# Plot group effects using sjPlot
plot_model(anova_model,
  type = "est",          # Plot estimates
  show.values = TRUE,
  value.offset = .3,
  title = "Effect of Dashboard Version on Total Score",
  axis.title = c("Dashboard Version", "Estimated Total Score"))
```

```
# Optional: plot means with confidence intervals (if you want a ggplot version)
library(ggplot2)
ggplot(df, aes(x = group, y = Total_Score)) +
  stat_summary(fun = mean, geom = "bar", fill = "skyblue", width = 0.6) +
  stat_summary(fun.data = mean_cl_normal, geom = "errorbar", width = 0.2) +
  labs(title = "Mean Total Score by Dashboard Version",
       x = "Dashboard Version",
       y = "Total Score") +
  theme_minimal()
```



power after pilot

```
# Load the power analysis library
p_load(pwr)

# Fit your model
model <- lm(Total_Score ~ group, data = df_final)

# Get summary object
model_summary <- summary(model)

# Extract estimate and standard error for groupImprovedBoth
coef_table <- model_summary$coefficients
estimate_both <- coef_table["groupImprovedBoth", "Estimate"]

# Get residual standard error (pooled SD)
pooled_sd <- model_summary$sigma

# Compute Cohen's d for the contrast (ImprovedBoth vs. reference group)
cohens_d <- estimate_both / pooled_sd

# Show the result
print(paste("Cohen's d:", round(cohens_d, 3)))
```

```
## [1] "Cohen's d: 0.322"
```

```
# Compute required sample size for 80% power, two-tailed test
pwr.t.test(d = cohens_d,
           power = 0.80,
           sig.level = 0.05,
           type = "two.sample",
           alternative = "two.sided")
```

```
##
##      Two-sample t test power calculation
##
##              n = 152.0925
##              d = 0.3222912
##      sig.level = 0.05
##      power = 0.8
##      alternative = two.sided
##
## NOTE: n is number in each group
```

2x2 groups

```
# Create new variables for hierarchy and chartjunk
df_valid$hierarchy <- ifelse(df_valid$group %in% c("ImprovedHierarchy", "ImprovedBoth"), "Improved", "Base")
df_valid$chartjunk <- ifelse(df_valid$group %in% c("ImprovedChartjunk", "ImprovedBoth"), "Improved", "Base")

# Convert to factors
df_valid$hierarchy <- factor(df_valid$hierarchy, levels = c("Base", "Improved"))
df_valid$chartjunk <- factor(df_valid$chartjunk, levels = c("Base", "Improved"))

# Check the result
table(df_valid$hierarchy, df_valid$chartjunk)
```

```
##
##      Base Improved
## Base      136      110
## Improved  127      135
```

```
# Run the 2x2 ANOVA with interaction
anova_model <- aov(Total_Score ~ hierarchy * chartjunk, data = df_valid)

# Summary of the ANOVA
print(summary(anova_model), digits = 3)
```

```
##              Df Sum Sq Mean Sq F value Pr(>F)
## hierarchy      1    37    36.6   10.33 0.0014 **
## chartjunk      1     1     0.7    0.20 0.6536
## hierarchy:chartjunk 1     4     4.3    1.21 0.2714
## Residuals    504   1786     3.5
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```

# Summary of the ANOVA
print(summary.lm(anova_model), digits = 3)

##
## Call:
## aov(formula = Total_Score ~ hierarchy * chartjunk, data = df_valid)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -3.606 -1.504 -0.164  1.103  5.836
##
## Coefficients:
##                                Estimate Std. Error t value Pr(>|t|)
## (Intercept)                   3.897      0.161    24.14  <2e-16 ***
## hierarchyImproved              0.709      0.232     3.05  0.0024 **
## chartjunkImproved             0.267      0.241     1.10  0.2699
## hierarchyImproved:chartjunkImproved -0.369      0.335    -1.10  0.2714
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.88 on 504 degrees of freedom
## Multiple R-squared:  0.0228, Adjusted R-squared:  0.017
## F-statistic: 3.92 on 3 and 504 DF,  p-value: 0.00879

```

```

# assuming anova_model is your 2x2 ANOVA (aov/lm or afex object)
library(effectsize)

```

```

library(effectsize)

```

```

round_df <- function(x, k = 3) {
  x <- as.data.frame(x)
  num <- vapply(x, is.numeric, logical(1))
  x[num] <- lapply(x[num], round, k)
  x
}

```

```

es_omega <- omega_squared(anova_model)
es_eta_p <- eta_squared(anova_model, partial = TRUE)
es_eta_g <- eta_squared(anova_model, generalized = TRUE)

```

```

round_df(es_omega)

```

```

##           Parameter Omega2_partial    CI CI_low CI_high
## 1      hierarchy           0.018 0.95  0.004      1
## 2      chartjunk           0.000 0.95  0.000      1
## 3 hierarchy:chartjunk       0.000 0.95  0.000      1

```

```

round_df(es_eta_p)

```

```

##           Parameter Eta2_partial    CI CI_low CI_high
## 1      hierarchy           0.020 0.95  0.005      1
## 2      chartjunk           0.000 0.95  0.000      1
## 3 hierarchy:chartjunk       0.002 0.95  0.000      1

```

```
round_df(es_eta_g)
```

```
##           Parameter Eta2_generalized    CI CI_low CI_high
## 1      hierarchy           0.020 0.95  0.005      1
## 2      chartjunk           0.000 0.95  0.000      1
## 3 hierarchy:chartjunk       0.002 0.95  0.000      1
```

```
means <- aggregate(Total_Score ~ hierarchy + chartjunk, data = df_valid, mean)
means$Total_Score <- round(means$Total_Score, 2)
means
```

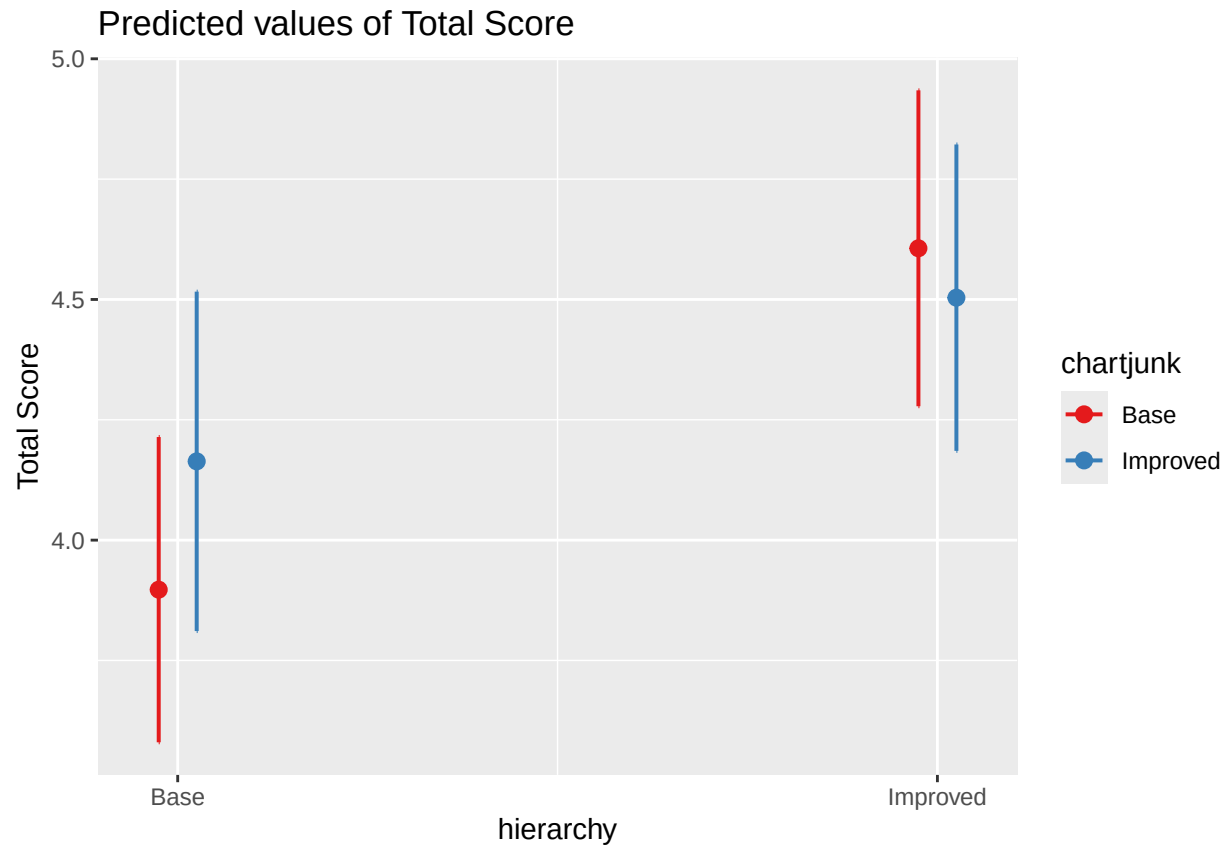
```
## hierarchy chartjunk Total_Score
## 1      Base      Base      3.90
## 2 Improved      Base      4.61
## 3      Base Improved      4.16
## 4 Improved Improved      4.50
```

```
# Use lm instead of aov for compatibility with sjPlot
lm_model <- lm(Total_Score ~ hierarchy * chartjunk, data = df_valid)

# Plot model using sjPlot
# install.packages("sjPlot") # Uncomment if not installed
library(sjPlot)

plot_model(lm_model, type = "int") # This shows interaction effects
```

```
## Ignoring unknown labels:
## * linetype : "chartjunk"
## * shape : "chartjunk"
```

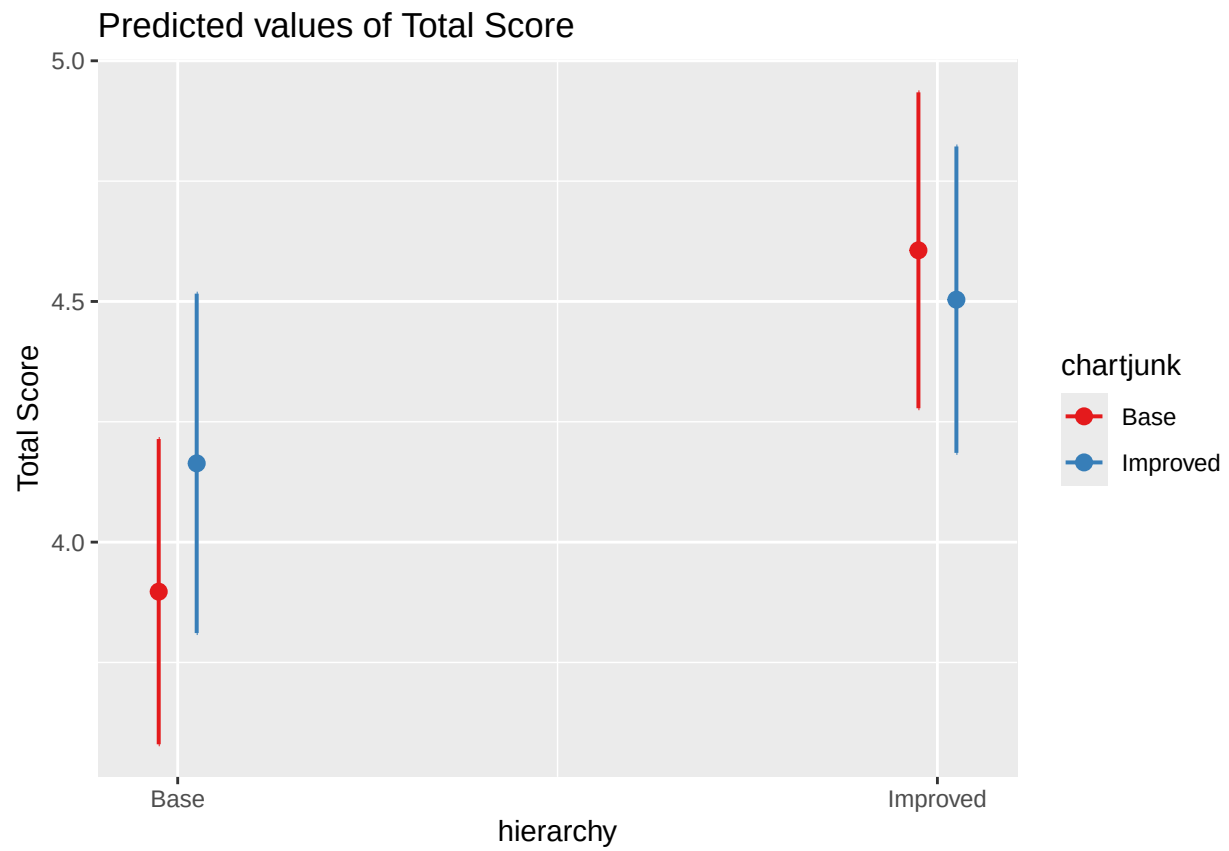


```
p_load(effects)
# Fit the model with lm for full compatibility
lm_model <- lm(Total_Score ~ hierarchy * chartjunk, data = df_valid)

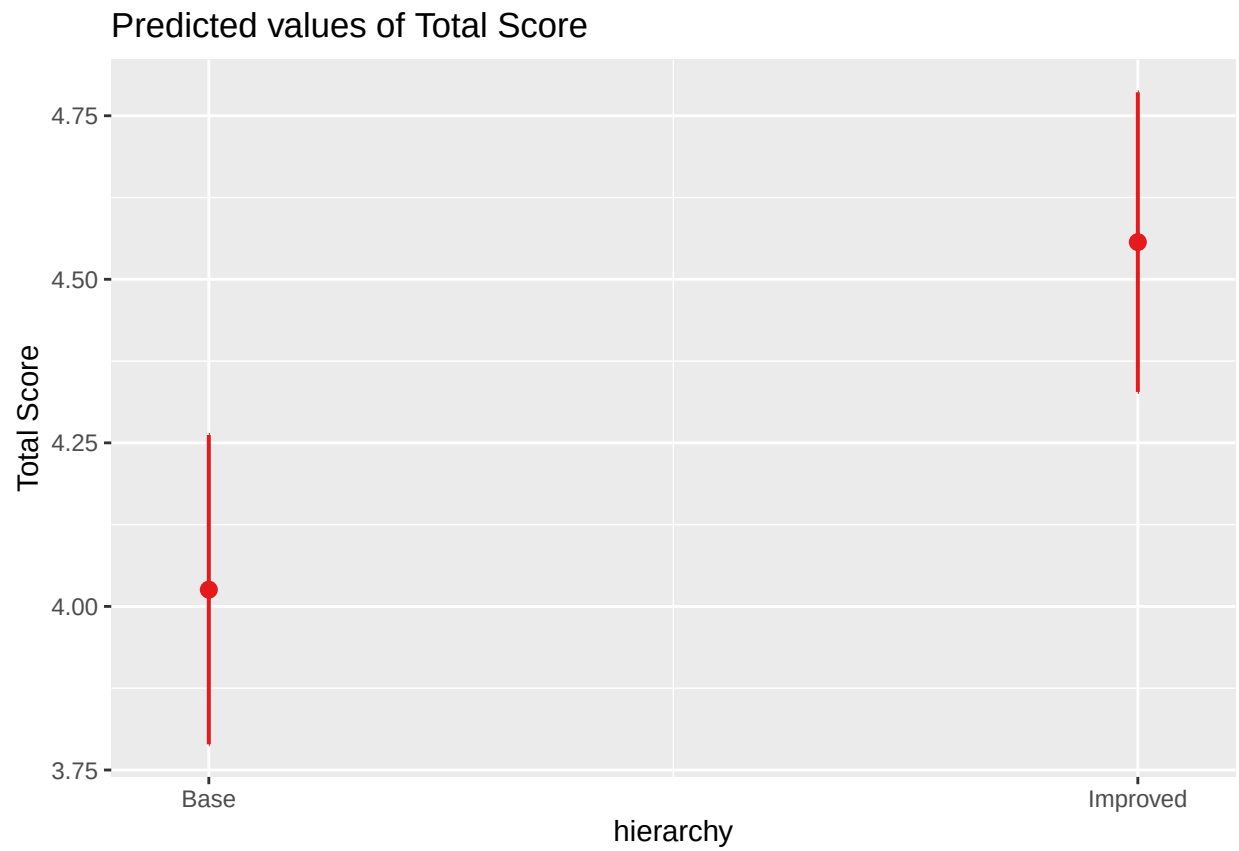
# # Show marginal effects for one factor
# plot_model(lm_model, type = "eff", terms = "hierarchy")
#
# # You can also show both factors:
# plot_model(lm_model, type = "eff", terms = c("hierarchy", "chartjunk"))

# Or the interaction:
plot_model(lm_model, type = "int", terms = c("hierarchy", "chartjunk"))

## Ignoring unknown labels:
## * linetype : "chartjunk"
## * shape : "chartjunk"
```



```
plot_model(lm_model, type = "eff", terms = c("hierarchy"))
```



```
summary.lm(anova_model)$r.squared
```

```
## [1] 0.02277723
```